

Feature Stores on Google Cloud

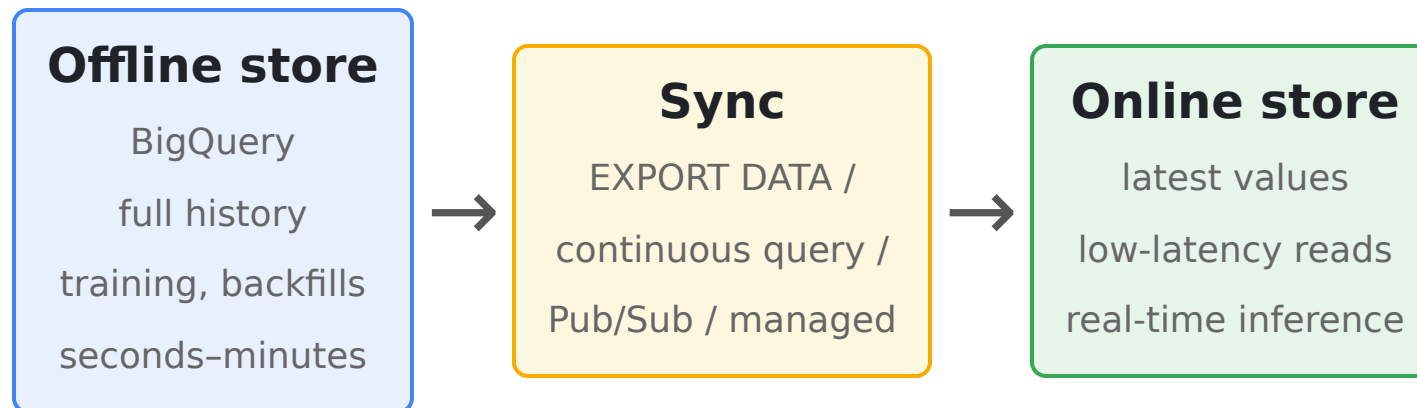
5 approaches — from zero infrastructure to sub-millisecond serving

BigQuery · Spanner · Bigtable · Valkey · Vertex AI Feature Store

45 notebooks · one shared 130K-entity dataset · [github.com/statmike/vertex-ai-mlops/MLOps/Feature Store](https://github.com/statmike/vertex-ai-mlops/MLOps/FeatureStore)

The Feature Store Problem

Features are **computed in batch** (BigQuery — seconds to minutes) but **served in real time** (milliseconds or less, per prediction). An online store bridges the two, and the offline store stays the source of truth.



The recurring questions: how fresh must features be? how fast must reads be? do you need to filter, JOIN, or transact at serve time? how much infrastructure do you want to own? Each approach answers these differently.

Five Approaches, One Offline Store

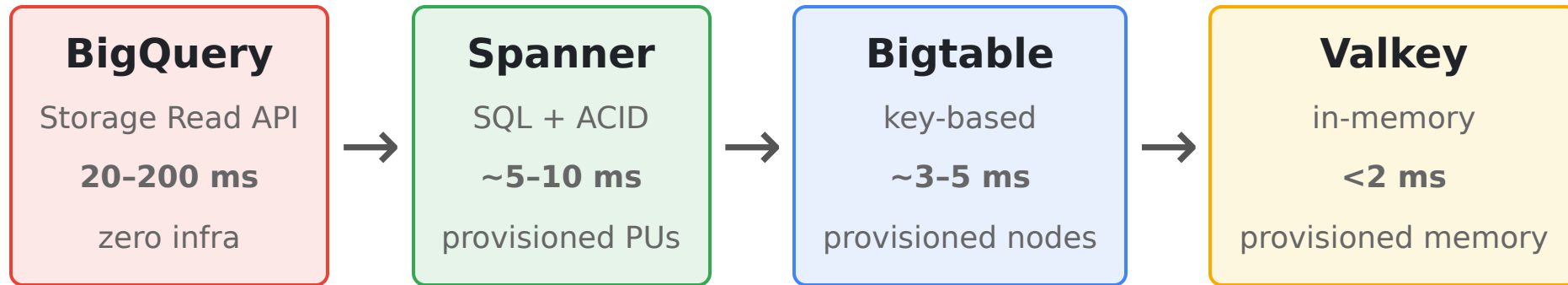
Every approach uses **BigQuery as the offline analytical engine**. They differ in the *online* path. Four are self-built on the **same 130K-entity dataset** (26 groups × 5,000, 224 columns) so they compare directly; the fifth is fully managed.

Approach	Online store	Notebooks
BigQuery	none — serve from BigQuery storage	3
Spanner	Cloud Spanner (SQL database)	12
Bigtable	Cloud Bigtable (wide-column)	15
Valkey	Memorystore for Valkey (in-memory)	9
Vertex AI FS	managed (Bigtable-backed or Optimized)	6

The first four trade latency for operational simplicity along a spectrum. Vertex AI sits apart — it *abstracts* the online store rather than exposing it.

The Latency Spectrum

The central trade-off: **lower latency costs more infrastructure and more constraints.**

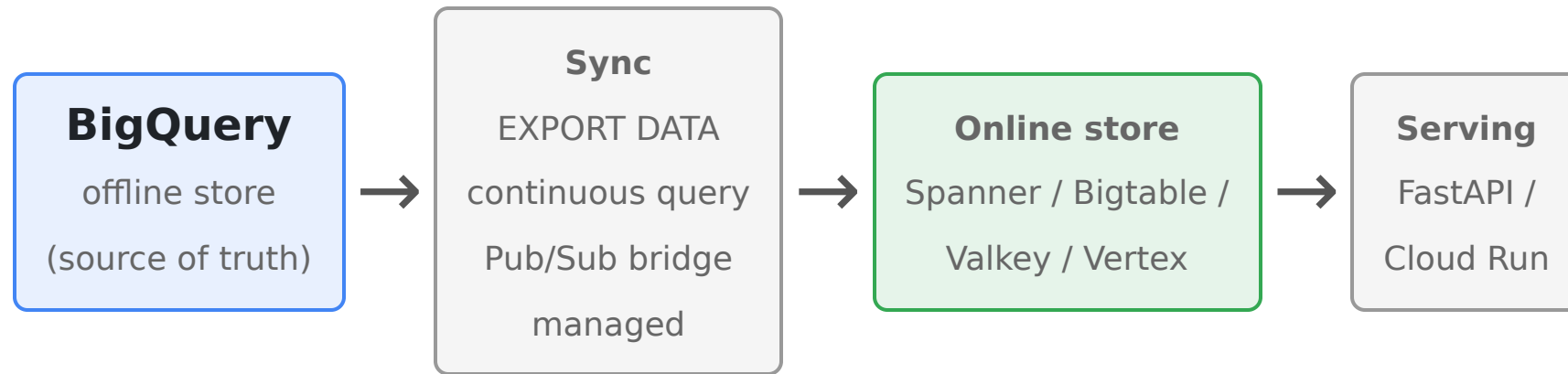


Vertex AI Feature Store — managed serving (Bigtable-backed or Optimized), off the spectrum: Google runs the infra and the registry

Latency = measured single-entity point read on the shared dataset. Faster reads buy you less: data must fit in memory (Valkey), or you give up SQL/JOINS (Bigtable).

Shared Architecture

One offline store, four online paths, plus direct writes for real-time signals.



Direct write path — apps write real-time signals straight to the online store (counters, state, quotes), bypassing BigQuery

The shared dataset: `dense_features` (130,000 rows × 224 columns, every BQ type) + `sparse_events` (2.6M rows). BigQuery FS serves it in place; the others sync a copy. **BigQuery stays the durable record everywhere.**

BigQuery Feature Store — Zero Infrastructure

Serve features **directly from BigQuery** — no online store, no sync, nothing to provision.

How — [Storage Read API](#) reads Colossus directly over gRPC, streaming Apache Arrow (zero-copy). Session caching + clustering.

Unique

- Always fresh (no sync lag)
- Filter on **any column**, ad-hoc
- Time travel built in
- Dataplex governance for free

Measured

- ~150–400 ms warm (2.7× via session pool)
- Arrow→numpy 235 µs (zero-copy)
- Bulk: wins vs Bigtable at 100+ entities

Cost — **\$0 infra**, 300 TiB/mo free read tier

Spanner Feature Store — SQL at the Serving Layer

Make the online store a **real SQL database** — the only approach with serving-time JOINS and multi-row ACID.

How — composite PK

`(entity_group, entity_id)` ;
`execute_sql` / `read()` ; FastAPI +
session pool. Enterprise edition for
vector indexes.

Unique

- **Secondary indexes** — query any column
- **Serving-time JOINS** (1/2/3-table)

Measured (100 PU)

- point read ~10 ms (strong), ~8 ms (stale)
- index: `rows_scanned` 1 vs 130,000
- concurrency: 10 writers, 0 lost updates
- SQL-filtered vector KNN ~45 ms

Cost — provisioned PUs (1000 = 1 node) + autoscaling; Enterprise for

Bigtable Feature Store — Key-Based, Full Control

Direct ownership of a sub-5 ms wide-column serving layer with atomic real-time writes.

How — read by row key; `#schema` metadata row decodes bytes; column families + cell versioning. 5 serialization strategies.

Unique

- **Atomic** read-modify-write, check-and-mutate
- 4 sync tiers (one-time → streaming)
- Multi-cluster replication: app-

Measured

- end-to-end ~5 ms (read 3 + decode 1 + predict 1)
- atomic counter RMW ~5 ms
- streaming aggregation ~50 ms

Cost — provisioned nodes, ~\$500+/mo (1-node min); bills continuously

Valkey Feature Store — Sub-Millisecond In-Memory

The fastest reads in the set, with native HNSW vector search and atomic counters.

How — [Memorystore for Valkey](#),
Cluster-Disabled; features as Redis
hashes `features:{group}:{id}` ;
`HGET` / `HMGET` ; `ConnectionPool`.

Unique

- **<2 ms reads** (only in-memory store)
- **TTL-on-sync** — stale data self-evicts → fall back to BigQuery
- Atomic `HTNCBBY` ≤1 ms: Lua

Measured (130K)

- point read ~1.6 ms; pipelined 0.12 ms/entity
- HNSW ~2 ms @ ~89% recall@10 vs FLAT ~7 ms @ 100%
- load 130K in ~25 s, ~300 MB

Cost — provisioned memory (hard ceiling); HA tier primary + replica

Vertex AI Feature Store — Fully Managed

Google runs the serving infra, the sync, and a formal feature registry.

How — managed online store:

Bigtable-backed (scalable) or

Optimized (ultra-low latency).

Serving unit is a **Feature View**;

`OnlineStoreReader` helper for async reads.

Unique

- The only **fully managed** option
- **Feature registry**: Feature Groups → Features → Feature Views

Vector search — ANN or brute-force, with filter columns + crowding, built in.

Note — defines its own source-table shapes (entity rows or history tables); **not** built on the shared 130K dataset like the other four.

Sync Patterns Compared

How features get from BigQuery to the online store. **Freshness costs complexity.**

Approach	Mechanism	Freshness	Notes
BigQuery	none — served in place	always fresh	no pipeline at all
Spanner	<code>EXPORT DATA format=CLOUD_SPANNER ;</code> <code>INSERT OR UPDATE</code> upsert	seconds (continuous)	continuous query needs Enterprise reservation; change streams for reverse sync
Bigtable	<code>EXPORT DATA</code> (CLOUD_BIGTABLE); 4 tiers	hours → ms	one-time / scheduled / continuous (<code>APPENDS()</code>) / streaming Pub/Sub
Valkey	Pub/Sub bridge (no native export)	seconds	continuous query → Pub/Sub → subscriber → <code>HSET</code> ; dead-letter + idempotency
Vertex AI	managed sync	managed	Google handles it

Spanner and Bigtable are **native** `EXPORT DATA` **targets**; Valkey builds the bridge itself; BigQuery needs no sync; Vertex hides it.

Direct Write & Transaction Patterns

For real-time signals written *outside* the batch pipeline (counters, quotes, state).

Approach	Write primitives	Atomicity
BigQuery	Storage Write API (append/DML)	n/a — read-serving layer
Spanner	DML, <code>INSERT OR UPDATE</code> , partitioned DML	multi-row / multi-table ACID (serializable)
Bigtable	single-row write, batch mutations, read-modify-write, check-and-mutate	single-row atomic
Valkey	<code>HSET</code> , <code>HINCRBY</code> / <code>HINCRBYFLOAT</code> , <code>MULTI/EXEC</code> , <code>WATCH</code> , Lua	single-key atomic (<1 ms)
Vertex AI	offline-first (write to BigQuery source)	n/a

Only **Spanner** spans rows and tables atomically. **Bigtable** and **Valkey** give the fastest single-entity atomic updates. **BigQuery/Vertex** are write-through-the-offline-store.

Vector Search Compared

All four self-built stores now have **native** vector search — they differ on index type, pre-filtering, and latency.

Approach	Index	Functions	Pre-filtering	Latency (130K)
BigQuery	IVF / TreeAH (or numpy)	VECTOR_SEARCH / numpy	SQL pushdown	~ms after bulk read
Spanner	ScaNN ANN + exact	APPROX_COSINE_DISTANCE	full SQL WHERE	~10-50 ms (filter+KNN, one query)
Bigtable	brute-force	COSINE_DISTANCE	row key only	scales with table size
Valkey	HNSW / FLAT	FT.SEARCH KNN	TAG + NUMERIC	~2 ms @ ~89% recall
Vertex AI	ANN / brute-force	managed	filter cols + crowding	managed

Spanner for vector search inside a SQL query; **Valkey** for fastest standalone ANN; **BigQuery** for offline scale.

Latency & Cost Comparison

Measured point-read latency and the cost model, side by side.

	BigQuery	Spanner	Bigtable	Valkey	Vertex AI
Single-entity read	20–200 ms	~5–10 ms	~3–5 ms	<2 ms	single-digit ms / sub-ms
Bulk (100+ entities)	strong (Storage API)	SQL IN / JOIN	batched reads	pipelined 0.12 ms/ea	managed
Query by any column	yes (SQL)	yes (indexes)	no (key only)	tag/numeric	feature-view
Multi-row transactions	no	ACID	single-row	single-key	no
Infrastructure	\$0 serverless	provisioned PUs	nodes (~\$500+/mo)	provisioned memory	managed
Scales to	petabytes	petabytes	petabytes	RAM ceiling	managed
Availability	BQ SLA	99.99% / 99.999%	multi-cluster	HA tier	managed SLA

Decision Framework

Start with latency + query needs:

- **20-200 ms OK, want simplest path?**
→ **BigQuery** (zero infra, any-column filter, governance)
- **Need SQL filters / JOINS / ACID at serve time?**
→ **Spanner**
- **Need sub-5 ms key lookups + atomic writes + full control?**
→ **Bigtable**

- **Need <2 ms reads or real-time counters?**
→ **Valkey** (if data fits in memory)
- **Want it all managed, with a feature registry?**
→ **Vertex AI Feature Store**

Then check the deciders: multi-region strong consistency → Spanner; vector search *inside* SQL → Spanner; fastest standalone ANN → Valkey; petabyte key-value → Bigtable.

When NOT to Use Each

A quick gut-check against the common failure modes.

Approach	Reconsider if...
BigQuery	you need single-digit-ms reads, or atomic real-time writes
Spanner	you need sub-3 ms reads, or want zero provisioned infra
Bigtable	you need to filter/JOIN on non-key columns, or want serverless
Valkey	the working set won't fit in RAM, or you need durable-by-default storage
Vertex AI	you need direct control over serialization, schema, or the serving layer

Every store keeps **BigQuery as the durable source of truth** — so you can change your mind, or run more than one, without losing data.

Explore the Series

BigQuery (3) · **Spanner** (12) · **Bigtable** (15) · **Valkey** (9) · **Vertex AI** (6)

Each series: a per-approach overview deck + hands-on notebooks, all on BigQuery as the offline store.

45 notebooks · [github.com/statmike/vertex-ai-mlops/MLOps/Feature Store](https://github.com/statmike/vertex-ai-mlops/MLOps/Feature%20Store)